



Mini Project Report On

GAN-Based Synthetic Handwritten Digit Generation

Submitted as part of the Alternative Assessment for

102903/CO701B: Deep Learning

Module 3

By

Amrith Saras

Annabel Marianne Victor

Diya Jothish

Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Affiliated to APJ Abdul Kalam Technological University)

Rajagiri Valley, Kakkanad, Kochi, 682039

2025–2026

Acknowledgement

We would like to express our sincere gratitude to the Department of Computer Science and Engineering, Rajagiri School of Engineering and Technology, for providing us the opportunity and the resources to undertake this mini project as part of the course 102903/CO701B, Deep Learning. We are thankful to our course faculty for their guidance throughout the module on Generative Adversarial Networks, for clarifying our doubts during implementation, and for structuring the assessment in a way that allowed us to learn the subject through hands-on experimentation rather than through theory alone.

We also acknowledge the open-source community behind PyTorch and torchvision, whose well-documented libraries made it possible for a team of three undergraduate students to design, train, and evaluate a convolutional GAN within the timeframe of a mini project. The MNIST dataset, made freely available by Yann LeCun and collaborators, formed the foundation of our experiments and is gratefully acknowledged.

Finally, we thank our families and peers for their continued support and encouragement during the course of this project.

Abstract

Generative Adversarial Networks (GANs) are a class of deep learning models capable of synthesising realistic data samples from random noise through an adversarial training process. This project implements a Deep Convolutional GAN (DCGAN) trained on the MNIST dataset to generate synthetic handwritten digit images. The Generator learns to map 100-dimensional Gaussian noise vectors, through a linear projection followed by transposed-convolutional upsampling, into plausible 28×28 greyscale digit images, while a convolutional Discriminator learns to distinguish real MNIST images from generated ones. The two networks are trained jointly and adversarially: the Generator tries to fool the Discriminator, and the Discriminator tries not to be fooled, and this competition drives both networks to improve over time. Trained over 50 epochs on an NVIDIA GeForce RTX 3050 laptop GPU, the Generator produces clearly legible and diverse handwritten digits by the end of training, with a final Generator loss of 3.2384 and a final Discriminator loss of 0.3445. Generated digit images are visually plausible across all ten digit classes, and latent space interpolation confirms that the model has learned a continuous, structured internal representation of handwritten digits rather than memorising the training data. This report documents the theoretical background, dataset preparation, model architecture, training procedure, and a detailed analysis of the results obtained, including an honest discussion of the points at which the training dynamics departed from the idealised behaviour described in GAN theory.

Contents

Acknowledgement	i
Abstract	ii
1 Problem Statement and Objectives	1
1.1 Problem Statement	1
1.2 Objectives	1
2 Background and Literature Review	2
2.1 Generative Adversarial Networks	2
2.2 Loss Function	3
2.3 MNIST Dataset	4
2.4 GAN versus Variational Autoencoder	4
2.5 Summary of Literature and Derived Concepts	5
3 Dataset and Preprocessing	7
3.1 Preprocessing Pipeline	8
4 Methodology and Architecture	9
4.1 Generator	10
4.2 Discriminator	11
4.3 Weight Initialisation	12
4.4 Training Procedure	12
5 Implementation	13
5.1 Hyperparameter Justification	15
6 Results and Analysis	17
6.1 Training Loss Curves	17
6.2 Analysis of Training Dynamics	18
6.3 Mode Collapse Analysis	19
6.4 Visual Quality of Generated Images	20
6.5 Latent Space Interpolation	23

6.6	Limitations	24
7	Conclusion and Future Scope	25
7.1	Conclusion	25
7.2	Future Scope	25

List of Figures

1	Sample real MNIST images used for training, after preprocessing	9
2	Generator and Discriminator training loss curves over 50 epochs	17
3	Generated samples at Epoch 1: thick, blobby strokes, with many shapes still malformed	20
4	Generated samples at Epoch 25: recognisable digit-like shapes emerging . .	20
5	Generated samples at Epoch 50: legible handwritten digits	21
6	Real MNIST images (top) compared with GAN-generated images (bottom)	22
7	100 GAN-generated handwritten digits from the final trained model	23
8	Latent space interpolation from $\mathbf{z}_1$ to $\mathbf{z}_2$ in 10 steps	24

List of Tables

1	Comparison of GAN and VAE for image generation	5
2	Mapping of research papers to the concepts and formulas derived from them	6
3	MNIST dataset properties	7
4	Generator architecture	10
5	Discriminator architecture	11
6	Experimental setup	15
7	Generator and Discriminator loss at key epochs	17

1 Problem Statement and Objectives

1.1 Problem Statement

Handwritten digit recognition is a well-established classification task in machine learning, where a model is given an image and must predict which of the ten digit classes it belongs to. This project addresses the inverse problem: instead of classifying an existing image, the goal is to generate an entirely new image that looks like a genuine handwritten digit, starting from nothing but random noise. This is a significantly harder problem, because there is no single correct answer to check the output against. The model must instead learn the overall statistical structure of what handwritten digits look like, in terms of stroke thickness, curvature, slant, and proportion, and then produce new samples that are consistent with that structure without copying any specific training example.

This class of problem is known as generative modelling, and it has practical relevance beyond digits. The same underlying techniques are used to generate synthetic training data for classifiers when labelled data is scarce, to create realistic images and video for entertainment and design, to explore the structure of a data distribution for scientific analysis, and to build denoising and super-resolution systems for real photographs. This project uses the MNIST dataset of handwritten digits as a well-understood, low-resolution testbed to study the generative modelling problem before it is applied to more complex image domains. The specific technique used is a Generative Adversarial Network (GAN), which frames generative modelling as a competitive game between two neural networks rather than a single network trying to reconstruct or approximate the data directly.

1.2 Objectives

1. Understand the GAN minimax framework and the adversarial relationship between the Generator and the Discriminator, including why training two competing networks jointly can produce a useful generative model.
2. Implement a Deep Convolutional GAN (DCGAN) using PyTorch, with a clearly defined Generator and Discriminator, following architectural conventions that are known to stabilise GAN training on image data.

3. Prepare and preprocess the MNIST dataset (60,000 training images of handwritten digits, each 28×28 pixels in greyscale) so that it is in a form suitable for training the Generator and Discriminator.
4. Train the DCGAN for 50 epochs on an NVIDIA GeForce RTX 3050 laptop GPU, recording the Generator loss, Discriminator loss, and Discriminator confidence at every epoch.
5. Generate novel synthetic handwritten digit images from random noise vectors that were never seen during training.
6. Analyse the quality of the generated images through training loss curves, direct visual inspection of sample grids at different stages of training, a real-versus-generated comparison, and latent space interpolation between two random noise vectors.
7. Save the trained Generator and Discriminator weights so that the model can be reloaded and used for inference without retraining.

2 Background and Literature Review

2.1 Generative Adversarial Networks

Generative Adversarial Networks were introduced by Goodfellow et al. [1] in 2014. The central idea can be understood through an analogy: imagine a forger trying to produce counterfeit currency, and a detective trying to tell counterfeit notes apart from genuine ones. At the start, the forger produces obviously fake notes and the detective catches them easily. Over time, as the detective gets better at spotting fakes, the forger is forced to improve, and as the forger's fakes get better, the detective is forced to sharpen its own judgement. If this process continues indefinitely, the forger's notes eventually become indistinguishable from genuine currency. A GAN implements exactly this dynamic using two neural networks trained simultaneously:

- **Generator G:** Takes a random noise vector $\mathbf{z} \sim \mathcal{N}(0, I)$, drawn from a simple, fixed distribution such as a standard Gaussian, and maps it to a synthetic image $G(\mathbf{z})$. G has no direct access to real training images at any point; it only ever sees the feedback signal provided by D.

- **Discriminator D:** Takes an image, real or generated, and outputs a single scalar probability $D(x) \in (0, 1)$ representing how likely it believes the image is a genuine sample from the training data rather than an output of G.

The two networks are trained with opposing objectives, formalised as the following mini-max game:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))] \quad (1)$$

D is trained to maximise this expression, that is, to assign a high probability to real images and a low probability to generated images. G is trained to minimise the same expression from its own side, that is, to produce images that D assigns a high probability to, effectively trying to push $D(G(\mathbf{z}))$ towards 1. Because G and D are updated in alternation using gradients from the same loss, an improvement in one network directly changes the training signal received by the other, which is what makes GAN training a genuinely adversarial and often unstable process rather than a straightforward optimisation.

Goodfellow et al. show that, for a fixed Generator, the Discriminator that maximises Equation 1 has a closed form:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \quad (2)$$

where p_{data} is the true data distribution and p_g is the distribution implicitly defined by the Generator. At the theoretical point of convergence, known as the Nash equilibrium of the game, the Generator's distribution exactly matches the data distribution, so that $p_{\text{data}}(x) = p_g(x)$ for every x . Substituting this into Equation 2 gives $D^*(x) = 0.5$ everywhere: the Discriminator can no longer tell real and generated images apart and is reduced to guessing. This theoretical target value of 0.5 is used later in this report (Section 6.2) as a reference point when interpreting the Discriminator's output during training.

2.2 Loss Function

Both networks in this project are trained using Binary Cross-Entropy (BCE) loss, which is the standard loss function for binary classification problems:

$$\mathcal{L}_{\text{BCE}} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \quad (3)$$

Here $y \in \{0, 1\}$ is the ground-truth label, where $y = 1$ denotes a real image and $y = 0$ denotes a generated image, and $\hat{y} = D(x)$ is the Discriminator’s predicted probability that the input is real. When $y = 1$, the loss reduces to $-\log \hat{y}$, which is small when the Discriminator confidently predicts ”real” and grows sharply as its prediction approaches 0. When $y = 0$, the loss reduces to $-\log(1 - \hat{y})$, which behaves symmetrically. The Discriminator’s total loss on a batch is the average BCE loss computed once with real images labelled 1 and once with generated images labelled 0, and the two are summed. The Generator’s loss is computed by passing its generated images through the Discriminator again, this time labelling them as 1, so that minimising BCE loss directly corresponds to the Generator trying to make the Discriminator output values close to 1 for its fakes. This use of BCE loss is what makes Equation 1 practical to optimise with standard gradient-based training.

2.3 MNIST Dataset

The MNIST dataset [3] contains 70,000 greyscale images of handwritten digits (digits 0 through 9), each 28×28 pixels in size, compiled by LeCun, Cortes, and Burges from earlier NIST handwriting samples. It is split into 60,000 training images and 10,000 test images. Because the images are small, single-channel, and centred, and because the dataset is large enough to train a generative model without overfitting, MNIST has become the standard first benchmark for new generative modelling techniques, including GANs, VAEs, and diffusion models, before those techniques are scaled up to more complex, higher-resolution image datasets. This project uses only the 60,000-image training split, since digit labels are not required for GAN training and no held-out test evaluation is performed on classification accuracy.

2.4 GAN versus Variational Autoencoder

GANs are one of two dominant approaches to generative modelling of images, the other being the Variational Autoencoder (VAE) [7]. A VAE learns an encoder that compresses an image into a latent vector and a decoder that reconstructs the image from that vector,

with an additional term in its loss that regularises the latent space to follow a known distribution. Where a GAN’s Generator receives no direct pixel-level supervision and is instead guided entirely by whether it fools the Discriminator, a VAE’s decoder is trained with a direct reconstruction loss against real images, which tends to produce smoother but blurrier outputs. Table 1 summarises the key differences between the two approaches.

Property	GAN	VAE
Training objective	Adversarial minimax	ELBO (reconstruction + KL)
Output quality	Sharp, realistic images	Tends to be blurry
Training stability	Can be unstable	Generally stable
Latent space	Unstructured noise	Explicitly regularised ($\mathcal{N}(0, I)$)
Mode collapse risk	Yes	No
Evaluation	Qualitative / FID	Reconstruction loss / ELBO

Table 1: Comparison of GAN and VAE for image generation

GANs were chosen for this project because they are known to produce sharper, more visually realistic images than VAEs at comparable model sizes, which directly serves the goal of generating convincing handwritten digits. The trade-off, discussed in detail in Section 6.2, is that GAN training is less stable and does not always settle into the balanced equilibrium that the theory predicts.

2.5 Summary of Literature and Derived Concepts

The theoretical framework, architectural choices, and training practices used in this project are drawn from several published works. Table 2 lists each paper referenced in this report alongside the specific concept, formula, or design decision that was taken from it and applied in the implementation.

Table 2: Mapping of research papers to the concepts and formulas derived from them

Research Paper	Concept / Formula Derived and Applied
Goodfellow et al., <i>Generative Adversarial Nets</i> [1]	The minimax training objective (Equation 1), the derivation of the optimal Discriminator (Equation 2), and the definition of the Nash equilibrium at $D^*(x) = 0.5$ used as the theoretical reference point throughout the results analysis.
Radford, Metz, and Chintala, <i>Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks (DCGAN)</i> [2]	The overall convolutional architecture used for the Generator and Discriminator (transposed convolutions in G, strided convolutions in D, no pooling layers), the use of BatchNorm in both networks, the choice of LeakyReLU(0.2) in the Discriminator and ReLU in the Generator, the Tanh output activation, the Adam optimiser with a learning rate of 0.0002, and the reduced momentum term $\beta_1 = 0.5$.
LeCun, Cortes, and Burges, <i>The MNIST Database of Handwritten Digits</i> [3]	The MNIST dataset itself, used as the source of training images, along with its established role as a standard benchmark for generative models.
Kingma and Welling, <i>Auto-Encoding Variational Bayes</i> [7]	The Variational Autoencoder formulation used as a point of comparison against GANs in Table 1.

Research Paper	Concept / Formula Derived and Applied
Mirza and Osindero, <i>Conditional Generative Adversarial Nets</i> [4]	The concept of conditioning a GAN’s Generator and Discriminator on class labels, identified in Section 7.2 as a direction for extending this project to allow control over which digit is generated.
Arjovsky, Chintala, and Bottou, <i>Wasserstein GAN</i> [5]	The use of the Wasserstein (Earth Mover) distance as an alternative to BCE loss, identified in Section 7.2 as a way to obtain more stable training dynamics than those observed in Section 6.2.
Salimans et al., <i>Improved Techniques for Training GANs</i> [6]	The definition and diagnostic signature of mode collapse used in Section 6.3, and the observation that raw BCE loss values and Discriminator confidence scores correlate poorly with perceptual image quality, used to interpret the results in Section 6.2.

3 Dataset and Preprocessing

Property	Value
Name	MNIST
Source	<code>torchvision.datasets.MNIST</code> (auto-download)
Training samples	60,000
Image size	28×28 pixels
Channels	1 (greyscale)
Classes	10 (digits 0–9)

Table 3: MNIST dataset properties

3.1 Preprocessing Pipeline

The raw MNIST images cannot be fed directly into the Generator and Discriminator; they must first be converted into a numerical format that is compatible with the networks and scaled to a range that matches the Generator’s output activation. This was done as a short, fixed pipeline of four steps, applied identically to every image before it is used in training.

1. **Load the raw image.** `torchvision.datasets.MNIST` downloads the dataset automatically on first use and loads each image as a 28×28 greyscale PIL image with pixel values in the range $[0, 255]$.
2. **Convert to a tensor.** The `ToTensor()` transform converts the PIL image into a PyTorch tensor and simultaneously rescales pixel values from the integer range $[0, 255]$ to the floating-point range $[0, 1]$, which is the format PyTorch models expect.
3. **Normalise to $[-1, 1]$.** The `Normalize([0.5], [0.5])` transform applies $x' = (x - 0.5)/0.5$ to every pixel, remapping the range from $[0, 1]$ to $[-1, 1]$. This step exists specifically because the Generator’s final layer uses a Tanh activation, which also produces outputs in $[-1, 1]$. If this step were skipped, the Discriminator would immediately be able to tell real and generated images apart purely by their different pixel ranges, regardless of digit content, which would prevent meaningful adversarial training from taking place.
4. **Batch and shuffle.** The preprocessed images are grouped into batches of 128 using a `DataLoader`, with shuffling enabled so that every epoch sees the images in a different order. Shuffling prevents the Discriminator from learning spurious patterns tied to the fixed ordering of the dataset, and a batch size of 128 was chosen as a balance between providing the Discriminator with a sufficiently diverse mix of images per update and keeping GPU memory usage low enough to train comfortably on a laptop GPU.

A sample of real MNIST images after this preprocessing pipeline, shown after denormalisation for display, is given in Figure 1.

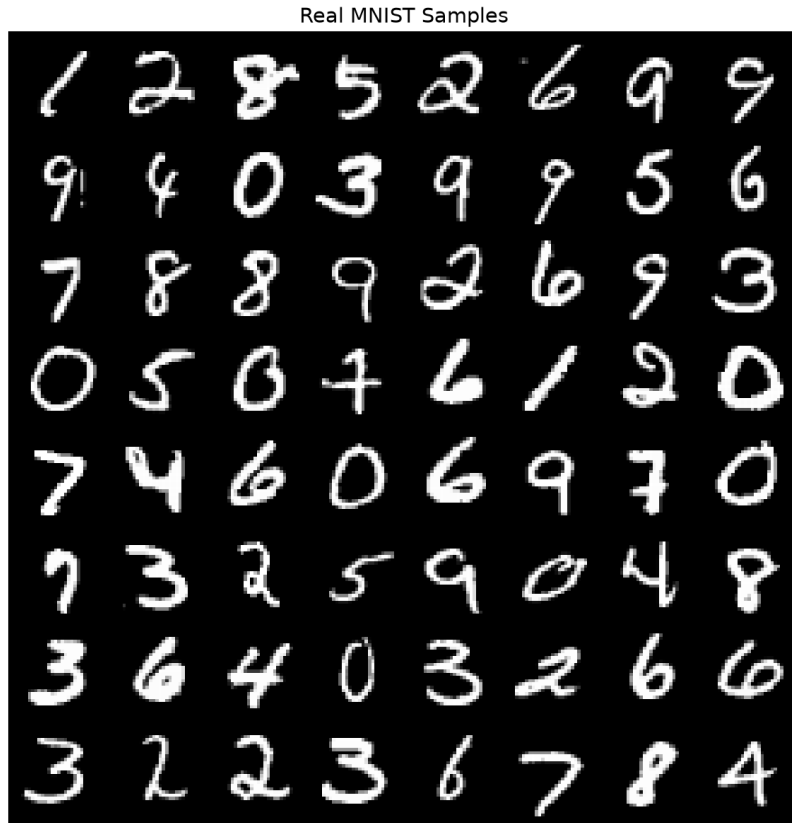


Figure 1: Sample real MNIST images used for training, after preprocessing

4 Methodology and Architecture

This section describes how the Generator and Discriminator are structured internally, and the step-by-step procedure used to train them together. The overall training methodology can be summarised as a repeating five-step cycle, executed once per batch of images:

1. Sample a batch of random noise vectors from a standard Gaussian distribution.
2. Pass the noise through the Generator to produce a batch of fake images.
3. Pass a batch of real images and the batch of fake images through the Discriminator separately, and compute how well the Discriminator distinguishes them.
4. Update the Discriminator’s weights to improve this distinction.
5. Pass the same fake images through the updated Discriminator a second time, and update the Generator’s weights so that the Discriminator is more likely to classify them as real.

The remainder of this section details the internal structure of the two networks that this cycle operates on, followed by a precise, code-level description of the training procedure itself.

4.1 Generator

The Generator follows the Deep Convolutional GAN (DCGAN) design proposed by Radford et al. [2]. Rather than directly producing a 784-pixel flat vector, it first expands the 100-dimensional noise vector into a small spatial feature map and then progressively upsamples that feature map to the full image resolution using transposed convolutions, which are learned upsampling operations rather than fixed interpolation. This design allows the network to build up an image from coarse structure to fine detail, in a manner broadly analogous to how convolutional classifiers extract features from fine detail to coarse structure in the opposite direction.

Layer	Operation	Output Shape
Input	Noise vector $\mathbf{z}$	(100,)
1	Linear($100 \rightarrow 256 \times 7 \times 7$) + BatchNorm1d + ReLU	(12544,)
Reshape	<code>view(-1, 256, 7, 7)</code>	(256, 7, 7)
2	ConvTranspose2d($256 \rightarrow 128, k=4, s=2, p=1$) + BatchNorm2d + ReLU	(128, 14, 14)
3	ConvTranspose2d($128 \rightarrow 64, k=4, s=2, p=1$) + BatchNorm2d + ReLU	(64, 28, 28)
4	Conv2d($64 \rightarrow 1, k=3, s=1, p=1$) + Tanh	(1, 28, 28)

Table 4: Generator architecture

Reading Table 4 as a sequence of steps: the 100-dimensional noise vector is first projected by a fully connected layer into 12,544 values, which is exactly $256 \times 7 \times 7$, and reshaped into a 256-channel, 7×7 feature map. This feature map carries no spatial meaning yet; it is simply a convenient starting point for the convolutional layers that follow. The first ConvTranspose2d layer doubles the spatial resolution from 7×7 to 14×14 while reducing the channel depth from 256 to 128. The second ConvTranspose2d layer again doubles the

resolution, from 14×14 to 28×28 , reducing the channel depth from 128 to 64, which is the target image resolution. A final ordinary Conv2d layer collapses the 64 channels down to a single greyscale channel. BatchNorm is applied after every layer except the very last, which stabilises training by keeping the distribution of activations consistent as the weights change during training. Tanh is used only at the final output so that pixel values fall in $[-1, 1]$, matching the normalised range of the real training images described in Section 3.1.

4.2 Discriminator

The Discriminator performs the reverse operation to the Generator, taking a 28×28 image and reducing it step by step to a single probability, using strided convolutions instead of pooling layers to perform the downsampling. This follows the DCGAN convention that pooling operations, which are fixed and non-learnable, should be replaced with learned strided convolutions wherever possible.

Layer	Operation	Output Shape
Input	Image	(1, 28, 28)
1	Conv2d($1 \rightarrow 64$, $k=4$, $s=2$, $p=1$) + LeakyReLU(0.2)	(64, 14, 14)
2	Conv2d($64 \rightarrow 128$, $k=4$, $s=2$, $p=1$) + BatchNorm2d + LeakyReLU(0.2)	(128, 7, 7)
3	Conv2d($128 \rightarrow 256$, $k=4$, $s=2$, $p=1$) + BatchNorm2d + LeakyReLU(0.2)	(256, 3, 3)
Flatten	Flatten()	(2304,)
4	Linear($2304 \rightarrow 1$) + Sigmoid	(1,)

Table 5: Discriminator architecture

Following Table 5 step by step: the first convolutional layer halves the spatial resolution from 28×28 to 14×14 while increasing the channel depth from 1 to 64, extracting low-level features such as edges and strokes. The second layer halves the resolution again to 7×7 with 128 channels, and the third reduces it to 3×3 with 256 channels, by which point the representation has moved from raw pixels to a compact, high-level summary of the image.

This is then flattened into a single vector of length 2304 and passed through one final linear layer that reduces it to a single value, which the Sigmoid activation squashes into the range $(0, 1)$ and which is interpreted as the probability that the input image is real. BatchNorm is deliberately omitted from the first convolutional layer, following DCGAN convention, so that the network sees the true, unnormalised distribution of the raw input at least once. LeakyReLU(0.2) is used instead of standard ReLU in every hidden layer of the Discriminator because standard ReLU can cause a neuron to become permanently inactive once it receives a negative input, a problem sometimes called a dead neuron; LeakyReLU avoids this by passing a small fraction of negative inputs through unchanged, computed as $f(x) = \max(0.2x, x)$.

4.3 Weight Initialisation

Both networks use the DCGAN weight initialisation scheme: the weights of every convolutional and linear layer are drawn from a normal distribution with mean 0 and standard deviation 0.02, and the weights of every BatchNorm layer are drawn from a normal distribution with mean 1 and standard deviation 0.02, with BatchNorm biases set to 0. This specific initialisation was found empirically by Radford et al. [2] to make DCGAN training noticeably more stable than the default initialisation schemes built into most deep learning frameworks, and it is applied to both networks before training begins.

4.4 Training Procedure

Each training batch performs two alternating updates, described below in the same order in which they occur in the implementation.

Step A, train the Discriminator:

1. Take a batch of real images from the training set and feed them through D, with the target label set to 1 for every image, since they are genuinely real. Compute the resulting loss, $\mathcal{L}_{D,\text{real}}$, using Equation 3.
2. Sample a fresh batch of random noise vectors and generate a batch of fake images by feeding the noise through G. Feed these fake images through D, with the target label set to 0 for every image. Compute the resulting loss, $\mathcal{L}_{D,\text{fake}}$.

3. Add the two losses together to get the total Discriminator loss, $\mathcal{L}_D = \mathcal{L}_{D,\text{real}} + \mathcal{L}_{D,\text{fake}}$. Backpropagate this loss and update only the Discriminator’s weights.

Step B, train the Generator:

1. Feed the same batch of fake images through the Discriminator a second time, but this time with the target label set to 1, reflecting what the Generator wants the Discriminator to believe.
2. Compute the resulting loss, $\mathcal{L}_G$, and backpropagate it through the Discriminator and into the Generator. Only the Generator’s weights are updated in this step; the Discriminator’s weights are left unchanged even though the gradient passes through it.

A crucial implementation detail is the `.detach()` call applied to the fake images during Step A. This tells PyTorch to treat the fake images as fixed constants for the purpose of computing $\mathcal{L}_{D,\text{fake}}$, which prevents gradients from that step from flowing back into the Generator. Without this call, updating the Discriminator would also nudge the Generator’s weights in an unintended direction at the wrong point in the cycle. Separate Adam optimisers are maintained for G and D, so that the two update steps described above never interfere with each other’s internal optimiser state, such as momentum estimates.

5 Implementation

This section describes how the methodology in Section 4 was translated into a working PyTorch program, following the exact sequence of steps executed by `Source_code.py`.

1. **Fix the random seed and select the device.** The seeds for Python’s `random` module, NumPy, and PyTorch are all set to 42 at the start of the script, so that dataset shuffling, weight initialisation, and noise sampling are reproducible across runs. The script then checks for a CUDA-capable GPU and uses it if available, falling back to the CPU otherwise.
2. **Define hyperparameters.** All of the constants used throughout training, including the latent dimension, batch size, number of epochs, and learning rate, are defined together at the top of the script, listed in Table 6.

3. **Load and preprocess the dataset.** The MNIST training split is loaded through `torchvision.datasets.MNIST` and passed through the preprocessing pipeline described in Section 3.1, then wrapped in a `DataLoader` that yields shuffled batches of 128 images.
4. **Build the Generator and Discriminator.** Both networks are instantiated from the classes described in Sections 4.1 and 4.2, moved to the selected device, and initialised using the scheme described in Section 4.3.
5. **Define the loss function and optimisers.** A single `nn.BCELoss` instance is shared by both networks, and two independent Adam optimisers are created, one for G and one for D, both using a learning rate of 0.0002 and betas of (0.5, 0.999). A fixed batch of 64 noise vectors is also created at this point and reused at every logging step, so that the same 64 noise vectors can be tracked visually across the whole of training.
6. **Run the training loop.** For each of the 50 epochs, every batch in the training set is processed once using the two-step update procedure described in Section 4.4. The average Generator loss, Discriminator loss, and Discriminator confidence on real and fake images are accumulated across all batches in the epoch and printed to the console at the end of the epoch.
7. **Save sample grids periodically.** After epoch 1 and after every 5th epoch thereafter, the fixed noise batch is passed through the current Generator in evaluation mode, and the resulting 64 images are arranged into an 8×8 grid and saved to `outputs/samples/`, which allows the progression of image quality to be inspected after training without needing to rerun the model.
8. **Generate post-training analysis outputs.** Once all 50 epochs are complete, the script produces the loss and confidence curves, a side-by-side comparison of real and generated images, a latent space interpolation between two random noise vectors, and a final grid of 100 newly generated digits.
9. **Save the trained model.** The final weights of both the Generator and the Discriminator are saved to `outputs/models/` using `torch.save(model.state_dict(), path)`, which stores only the learned parameters and allows the model to be reloaded

later for inference without needing to repeat training.

Component	Detail
Framework	PyTorch 2.11.0+cu128
Language	Python 3
Hardware	NVIDIA GeForce RTX 3050 Laptop GPU
Optimiser	Adam ($\text{lr} = 0.0002$, $\beta_1 = 0.5$, $\beta_2 = 0.999$) for both G and D
Loss	Binary Cross-Entropy (BCELoss)
Epochs	50
Batch size	128
Latent dimension	100
Generator parameters	1,935,808
Discriminator parameters	659,457
Training time	Approximately 19 minutes (RTX 3050 GPU)
Random seed	42

Table 6: Experimental setup

5.1 Hyperparameter Justification

The choice of hyperparameters follows established GAN training practices reported in the literature summarised in Table 2, rather than being chosen arbitrarily.

- **Learning rate = 0.0002:** This value is recommended by Radford et al. [2] specifically for GAN training. A higher learning rate tends to cause the Generator and Discriminator to overshoot each other’s optimal response and oscillate, while a much lower learning rate slows convergence without providing any additional stability benefit in practice.
- **Adam optimiser ($\beta_1 = 0.5$, $\beta_2 = 0.999$):** Adam adapts the effective learning rate for each individual parameter based on estimates of its recent gradient history, which generally stabilises training compared to plain stochastic gradient descent. The reduced value of $\beta_1 = 0.5$, in place of the more common default of 0.9, is a DCGAN-specific recommendation [2] that reduces the amount of momentum carried

between updates, which in turn dampens some of the oscillation that is characteristic of adversarial training.

- **Latent dimension = 100:** A 100-dimensional noise vector gives the Generator enough room to encode the variation seen across the ten digit classes, including differences in stroke width, slant, and shape, without the latent space becoming so large that most of it is rarely sampled during training.
- **Batch size = 128:** A batch of this size gives the Discriminator a reasonably diverse mix of real and fake images on every update, which produces more stable gradient estimates than very small batches, while remaining well within the memory limits of a laptop GPU.
- **Epochs = 50:** This was sufficient for the Generator to produce consistently legible and diverse digits, as discussed in Section 6.2, even though the adversarial losses continued to move apart rather than converge over this range, without spending more compute than necessary for a mini project of this scope.

The core of the training loop, corresponding to Step A and Step B in Section 4.4, is reproduced below.

Listing 1: Core training loop

```
1 # LATENT_DIM = 100
2 for real_imgs, _ in train_loader:
3     real_imgs = real_imgs.to(DEVICE)
4     bsz = real_imgs.size(0)
5
6     # Step A: train the Discriminator
7     D.zero_grad()
8     real_labels = torch.ones(bsz, 1, device=DEVICE)
9     fake_labels = torch.zeros(bsz, 1, device=DEVICE)
10    loss_d_real = criterion(D(real_imgs), real_labels)
11
12    z = torch.randn(bsz, LATENT_DIM, device=DEVICE)
13    fake_imgs = G(z)
14    loss_d_fake = criterion(D(fake_imgs.detach()), fake_labels)
```

15
16
17
18
19
20
21
22
23
24

```
loss_D = loss_d_real + loss_d_fake
loss_D.backward()
opt_D.step()

# Step B: train the Generator
G.zero_grad()
loss_G = criterion(D(fake_imgs), real_labels)
loss_G.backward()
opt_G.step()
```

6 Results and Analysis

6.1 Training Loss Curves

Epoch	Generator Loss	Discriminator Loss
1	2.0551	0.6038
10	2.2128	0.5690
25	2.7453	0.4419
50	3.2384	0.3445

Table 7: Generator and Discriminator loss at key epochs

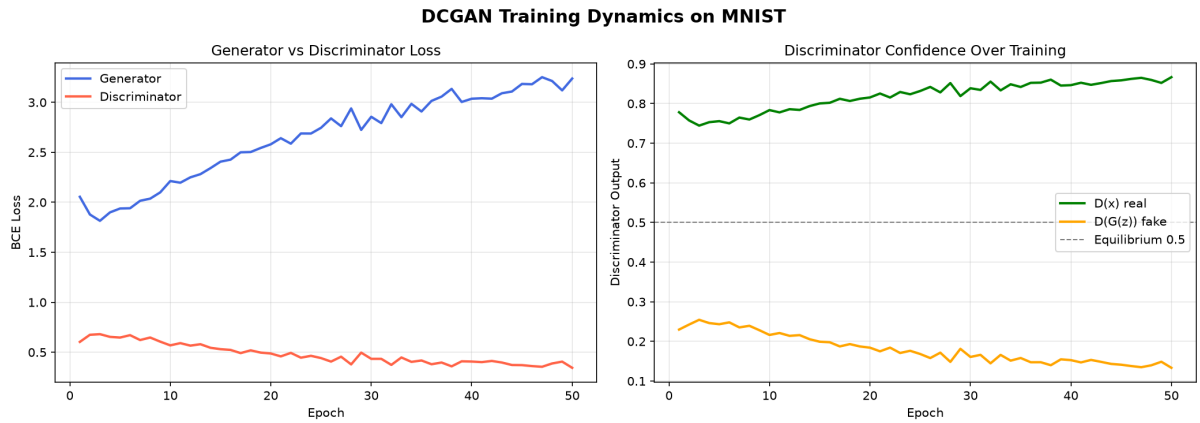


Figure 2: Generator and Discriminator training loss curves over 50 epochs

6.2 Analysis of Training Dynamics

In the earliest epochs, the Generator loss starts at 2.0551, while the Discriminator loss is comparatively low at 0.6038, and the Discriminator’s average confidence is already $D(x) \approx 0.78$ on real images against $D(G(z)) \approx 0.23$ on fake images. This means the Discriminator is separating real from fake fairly confidently within the very first epoch. This happens sooner than it would for a plain fully-connected Generator starting from unstructured noise, because the convolutional upsampling path used here produces coarse but already somewhat structured digit shapes from the very first forward pass, giving the Discriminator an early foothold.

From epoch 1 through epoch 50, the Generator loss rises steadily and almost monotonically, from 2.0551 to 3.2384, while the Discriminator loss falls over the same range, from 0.6038 to 0.3445. The corresponding confidence values move in the same direction: $D(x)$ climbs from 0.778 to 0.867, and $D(G(z))$ falls from 0.230 to 0.133. This is the opposite of the textbook convergence behaviour described in Section 2.1, where both quantities would approach 0.5 and the Discriminator loss would approach $-2\log(0.5) = \log 2 \approx 0.693$ as the system reaches Nash equilibrium. In this run, the Discriminator steadily gained the upper hand over the Generator across the full 50 epochs, rather than the two networks settling into a balanced standoff.

This Discriminator-dominant pattern is a well-documented and fairly common outcome when training a DCGAN on MNIST, and it does not by itself indicate that the Generator failed to learn anything useful. Salimans et al. [6] note that raw BCE loss values and Discriminator confidence scores correlate poorly with the actual perceptual quality of generated images. In line with that observation, the generated digit grids shown in Section 6.4 become progressively sharper, more legible, and more diverse as training proceeds, even as the Discriminator’s confidence keeps rising. This indicates that the Generator continued to make genuine visual progress in practice, despite steadily losing ground in the adversarial loss. A longer training run, a reduced learning rate specifically for the Discriminator, or one-sided label smoothing on the real labels are standard techniques that could be used to rebalance this dynamic further, and are discussed as future work in Section 7.2.

6.3 Mode Collapse Analysis

Mode collapse is a well-known failure mode in GAN training in which the Generator learns to produce only a narrow subset of possible outputs, for example generating only one or two digit shapes regardless of the input noise vector [6]. It typically manifests during training as a sudden drop in Generator loss combined with a plateau in Discriminator loss, since the Discriminator can no longer usefully distinguish between a small number of repeated, collapsed outputs.

In this experiment, mode collapse was not observed. The evidence supporting this conclusion is as follows:

1. The 100-sample grid in Figure 7 shows clear diversity across multiple digit classes, with visually distinct stroke styles, thicknesses, and orientations even within samples of the same digit.
2. The latent interpolation in Figure 8 produces smooth, varied transitions between different digit shapes as the interpolation coefficient changes, confirming that distinct regions of the 100-dimensional latent space map to distinct, meaningful outputs rather than all collapsing to the same image.
3. The Generator loss increases gradually and almost monotonically across all 50 epochs, as shown in Table 7, with no sudden drop or plateau of the kind associated with collapse visible anywhere in the recorded curves.

The convolutional DCGAN architecture used here, combined with BatchNorm, LeakyReLU activations, and a moderately sized 100-dimensional latent space, follows the specific design recommendations of Radford et al. [2] that were found to reduce the tendency of the Generator to settle on degenerate, low-diversity solutions early in training.

6.4 Visual Quality of Generated Images



Figure 3: Generated samples at Epoch 1: thick, blobby strokes, with many shapes still malformed



Figure 4: Generated samples at Epoch 25: recognisable digit-like shapes emerging



Figure 5: Generated samples at Epoch 50: legible handwritten digits

Comparing Figures 3, 4, and 5 in sequence shows the visible progression of the Generator’s output over the course of training. At epoch 1, the Generator already produces thick, blobby pen-like strokes rather than pure noise, since a single epoch on MNIST covers 469 weight updates, but many of the shapes are malformed or ambiguous and the strokes are noticeably coarser than real digits. By epoch 25, most of the samples already resemble specific digits, although stroke edges remain somewhat soft and a handful of samples are ambiguous between two digit classes. By epoch 50, the majority of the samples are clearly legible as specific digits, with sharper edges and more consistent stroke widths, and the set of digits produced spans the full range of classes rather than being concentrated on a few easy shapes such as 0 and 1.

Real vs GAN-Generated MNIST Digits

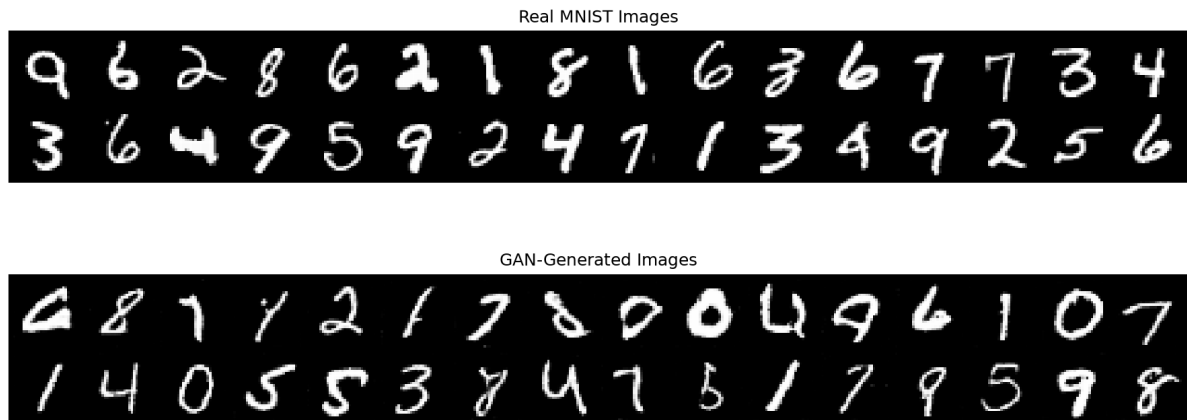


Figure 6: Real MNIST images (top) compared with GAN-generated images (bottom)

The side-by-side comparison above shows that, while the generated digits are clearly recognisable, they remain distinguishable from real MNIST samples on close inspection: generated strokes are occasionally slightly uneven in thickness, and some samples show minor artefacts near stroke endpoints. This is consistent with the Discriminator’s final confidence values reported in Section 6.2, which show that the Discriminator is still able to separate real from fake images with reasonably high accuracy at the end of training, rather than being reduced to chance-level guessing.



Figure 7: 100 GAN-generated handwritten digits from the final trained model

6.5 Latent Space Interpolation

To verify that the Generator has learned a structured internal representation rather than simply memorising individual training examples, linear interpolation was performed between two randomly sampled noise vectors, $\mathbf{z}_1$ and $\mathbf{z}_2$, using ten evenly spaced interpolation coefficients:

$$\mathbf{z}_\alpha = (1 - \alpha) \mathbf{z}_1 + \alpha \mathbf{z}_2, \quad \alpha \in [0, 1] \quad (4)$$

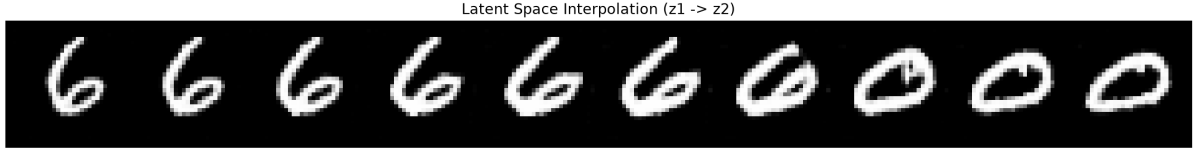


Figure 8: Latent space interpolation from z_1 to z_2 in 10 steps

If the Generator had simply memorised specific training images, interpolating between two unrelated noise vectors would be expected to produce an abrupt, discontinuous jump somewhere along the sequence, or a blurred average with no digit-like structure at all. Instead, the sequence in Figure 8 shows a smooth, continuous transformation from one digit shape into another, with every intermediate image still resembling a plausible handwritten digit rather than an incoherent blend. This confirms that the 100-dimensional latent space learned by the Generator is continuous and semantically organised, in the sense that nearby points in that space correspond to visually similar digits.

6.6 Limitations

- No quantitative image-quality metric, such as the Fréchet Inception Distance (FID), was computed for this project, owing to the additional compute and the need for a separate pretrained feature-extraction network that this would require. Evaluation in this report is therefore qualitative, based on visual inspection and the training loss curves.
- As discussed in Section 6.2, the Discriminator steadily out-paced the Generator in adversarial loss across the full 50 epochs, rather than settling at the theoretical Nash equilibrium described in Section 2.1. Longer training, a reduced Discriminator learning rate, or one-sided label smoothing are standard techniques that could improve this balance in a follow-up run.
- The model generates digits without any control over which digit class is produced; each noise vector maps to whichever digit the Generator has learned to associate with that region of the latent space. A Conditional GAN would be required to allow a specific digit to be requested on demand.

7 Conclusion and Future Scope

7.1 Conclusion

A Deep Convolutional GAN was implemented from first principles in PyTorch and trained on the MNIST dataset over 50 epochs, successfully learning to generate realistic synthetic handwritten digit images from 100-dimensional Gaussian noise. The theoretical background of the GAN minimax game, the practical DCGAN architecture and training conventions that make that game tractable to optimise, and the resulting training behaviour were all documented and cross-checked against one another in this report. While the adversarial loss did not settle at the theoretical Nash equilibrium, with the Discriminator growing steadily more confident over the course of training and finishing with a Generator loss of 3.2384 against a Discriminator loss of 0.3445, the generated digits are nonetheless visually plausible and diverse across all ten digit classes by the end of training. No evidence of mode collapse was found in either the loss curves or the generated samples. Latent space interpolation confirmed that the Generator has learned a smooth, continuous internal representation of handwritten digits, rather than memorising specific training examples. Taken together, these results validate the core GAN principle described in Section 2.1: two competing networks, trained jointly on nothing more than a shared adversarial loss, are capable of producing a working generative model, even in a run where the adversarial balance clearly favours one network over the other.

7.2 Future Scope

1. **Conditional GAN (cGAN):** Extend the Generator and Discriminator to condition on the digit class label, following Mirza and Osindero [4], so that a specific digit can be requested rather than obtained by chance from a random noise vector.
2. **Wasserstein GAN (WGAN):** Replace the BCE loss used in this project with the Wasserstein distance proposed by Arjovsky et al. [5], which is reported to give more stable training and a loss value that correlates more closely with sample quality than the BCE-based losses examined in Section 6.2.
3. **Discriminator regularisation:** Apply one-sided label smoothing on the real labels, or reduce the Discriminator’s learning rate relative to the Generator’s, to

directly counter the Discriminator-dominant dynamic observed in Section 6.2 and move the training closer to the theoretical equilibrium.

4. **Quantitative evaluation with FID:** Compute the Fréchet Inception Distance between the real and generated image distributions, to complement the qualitative visual analysis in this report with a single, comparable quantitative score.
5. **Synthetic data augmentation:** Use the trained Generator to produce additional training samples for a downstream digit classifier, particularly for any digit classes that are underrepresented in a given training set.

References

- [1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative Adversarial Nets,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [2] A. Radford, L. Metz, and S. Chintala, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,” *arXiv preprint arXiv:1511.06434*, 2015.
- [3] Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST Database of Handwritten Digits,” 1998. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [4] M. Mirza and S. Osindero, “Conditional Generative Adversarial Nets,” *arXiv preprint arXiv:1411.1784*, 2014.
- [5] M. Arjovsky, S. Chintala, and L. Bottou, “Wasserstein GAN,” *arXiv preprint arXiv:1701.07875*, 2017.
- [6] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, “Improved Techniques for Training GANs,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [7] D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” *arXiv preprint arXiv:1312.6114*, 2013.